UE04

Arrays & Algorithms

LEGAL NAME	*LEONARD-ADRIAN BUNEA*
PREFERRED NAME	ZOE BUNEA
TIME SPENT	04:05:44

Table of Contents

Table of Figures

Exercise 1 – Array Statistics

Solution Idea

This exercise requires the calculation of mean, median, minimum and maximum of an integer array.

Mean

Calculating the mean only requires calculating the sum and then dividing by the length of the array. Before dividing, either the sum or the length should be cast to a double to avoid integer division.

Median

Calculating the median is a bit more complicated, since it requires first sorting the array and depending on whether it's even or odd in length, it requires a different calculation.

If it's even, there are two mid-points, to get the second divide the array length by two and to get the first, subtract one from the second mid-point (since arrays are 0 indexed).

If it's odd, just divide the length by two and that's the index of the median. Since integer division rounds down, one can safely assume that you'll get the middle.

Minimum and Maximum

To get the minimum and maximum values, one should just loop through the array once, save the current minimum and maximum values and compare and swap until the loop finishes.

Code

Mean

```
public void calculateMean()
{
    int sum = 0;

    for (int num : getInputArray())
    {
        sum += num;
```

```
    }

    setMean((double) sum / getInputArray().length);
}
```

Figure 1: calculateMean

Median

```
public void calculateMedian()
{
    int[] sortedArray = Arrays.copyOf(getInputArray(),
getInputArray().length);
    Arrays.sort(sortedArray);

    if (sortedArray.length % 2 == 0)
    {
        int mid1 = sortedArray.length / 2;
        int mid2 = mid1 - 1;
        setMedian((sortedArray[mid1] + sortedArray[mid2]) /
2.0);
    }
    else
    {
        int mid = sortedArray.length / 2;
        setMedian(sortedArray[mid]);
    }
}
```

Figure 2: calculateMedian

Minimum and Maximum

```
public void calculateMinAndMax()
{
    int min = getInputArray()[0];
    int max = getInputArray()[0];

    for (int num : getInputArray())
    {
        if (num < min)
        {
```

```
            min = num;
        }
        if (num > max)
        {
            max = num;
        }
    }

    minimum = min;
    maximum = max;
}
```

Figure 3: calculateMinAndMax

Test

```
@Test
void testArrayFromExercise()
{
    excercise = new ArrayStatisticsExercise(new int[]{15, 8, 23,
4, 42, 11, 19});

    excercise.runExcercise();
    excercise.displayOutput();

    double expectedMean = 17.43;
    double expectedMedian = 15.0;
    int expectedMin = 4;
    int expectedMax = 42;

    assertEquals(expectedMean, excercise.getMean(), 0.01);
    assertEquals(expectedMedian, excercise.getMedian(), 0.001);
    assertEquals(expectedMin, excercise.getMinimum());
    assertEquals(expectedMax, excercise.getMaximum());
}
```

Figure 4: Statistics Test

```
=== Array Statistics ===
Arrays: [15, 8, 23, 4, 42, 11, 19]
Mean: 17.43
```

```
Median: 15.00
Minimum: 4
Maximum: 42
```

Figure 5: Statistics Test Output

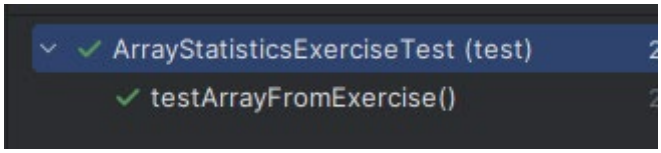

Figure 6: Successful Unit Test

Exercise 2 – Array Reversal

Solution Idea

Reversing an array in place requires following algorithm:

1. Save first array element in a temp variable
2. Set first array element to last
3. Set last array element to temp
4. Continue like this with second and second last and so on

This algorithm is shown in Figure 7.

Code

```
private int[] reverseArray(int[] array)
{
    int[] arrayCopy = Arrays.copyOf(array, array.length);

    int temp;

    for (int i = 0; i < arrayCopy.length / 2; i++)
    {
        temp = arrayCopy[i];
        arrayCopy[i] = arrayCopy[arrayCopy.length - 1 - i];
        arrayCopy[arrayCopy.length - 1 - i] = temp;
    }
```

```
    return arrayCopy;
}
```

Figure 7: reverseArray

Test

```
@Test
void testArrayReversalOdd()
{
    int[] inputArray = {10, 20, 30, 40, 50};

    ArrayReversalExercise excercise = new
ArrayReversalExercise(inputArray);

    excercise.runExcercise();

    int[] reversedArray = excercise.getReversedArray();

    int[] expectedArray = {50, 40, 30, 20, 10};

    for (int i = 0; i < expectedArray.length; i++)
    {
        assert(reversedArray[i] == expectedArray[i]);
    }
}
```

Figure 8: array reversal test

```
=== Array Reversal ===
Original Array: [10, 20, 30, 40, 50]
Reversed Array: [50, 40, 30, 20, 10]
```

Figure 9: array reversal test output

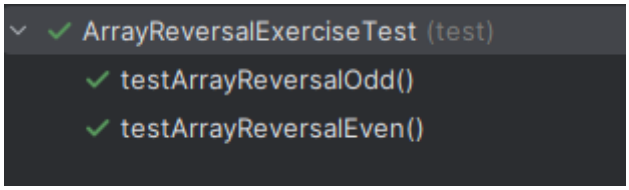

Figure 10: array reversal unit tests

Exercise 3 – Fuzzy Search

Solution Idea

Loop through the array and save the difference of the current index with the target value, if during another iteration the difference is smaller, save that index as the closest match instead. If the index value is the same as the target, simply return that.

Code

```
public record FuzzySearchResult(boolean foundExactMatch, int
bestMatchIndex, int bestMatchValue) { }
```

Figure 11: Fuzzy search result record

```
private FuzzySearchResult fuzzySearch(int target, int[] array)
{
    int difference = Integer.MAX_VALUE;
    int closestIndex = -1;

    for (int i = 0; i < array.length; i++)
    {
       if (array[i] == target)
       {
           return new FuzzySearchResult(true, i, array[i]);
       }

       int currentDifference = Math.abs(array[i] - target);

       if (currentDifference < difference)
       {
           difference = currentDifference;
           closestIndex = i;
       }
    }

    return new FuzzySearchResult(false, closestIndex,
```

```
array[closestIndex]);
}
```

Figure 12: Fuzzy Seach

Test

```
@Test
void testFuzzySearchNoMatchExists()
{
    int[] testArray = {15, 8, 23, 4, 42, 11, 19};
    int targetValue = 20;
    int expectedClosestValue = 19;
    int expectedClosestIndex = 6;

    FuzzySearchExercise fuzzySearchExercise = new
FuzzySearchExercise(testArray, targetValue);
    fuzzySearchExercise.runExcercise();
    fuzzySearchExercise.displayOutput();

    assertEquals(expectedClosestValue,
fuzzySearchExercise.getFuzzySearchResult().bestMatchValue());

assertFalse(fuzzySearchExercise.getFuzzySearchResult().foundExac
tMatch());
    assertEquals(expectedClosestIndex,
fuzzySearchExercise.getFuzzySearchResult().bestMatchIndex());
}
```

Figure 13: Fuzzy Search Test

```
=== Fuzzy Search ===
Array: [15, 8, 23, 4, 42, 11, 19]
Searching for: 20
Exact match not found.
Closest value: 19 at index 6
Difference: 1
```

Figure 14: Fuzzy Search Result

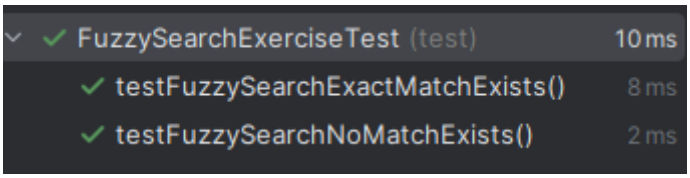

Figure 15: Successful unit tests

Exercise 4 – Array Merging

Solution Idea

One way to do this is to create a new array with the combined length of both arrays, loop through the first array and copy the values, save the index, then do the same with the second array. I just used System.arraycopy() instead.

Code

```
private int[] mergeArrays(int[] array1, int[] array2)
{
    int length1 = array1.length;
    int length2 = array2.length;
    int[] mergedArray = new int[length1 + length2];

    System.arraycopy(array1, 0, mergedArray, 0, length1);
    System.arraycopy(array2, 0, mergedArray, length1, length2);

    return mergedArray;
}
```

Figure 16: mergeArrays

Test

```
@Test
void testArrayMergingSmallArrays()
{
    int[] array1 = {10, 20, 30};
    int[] array2 = {40, 50, 60, 70};
    int[] expectedMergedArray = {10, 20, 30, 40, 50, 60, 70};

    ArrayMergingExercise excercise = new
ArrayMergingExercise(array1, array2);
```

```
    excercise.runExercise();
    excercise.displayOutput();
    int[] mergedArray = excercise.getMergedArray();

    assertArrayEquals(expectedMergedArray, mergedArray);
}
```

Figure 17: array merging test

```
=== Merging Two Arrays ===
Array 1: [10, 20, 30]
Array 2: [40, 50, 60, 70]
Merged array: [10, 20, 30, 40, 50, 60, 70]
```

Figure 18: array merging output

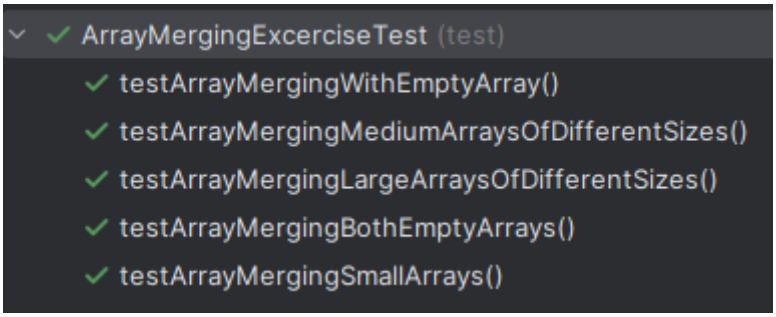

Figure 19: Array Merging Unit Tests

Exercise 5 – Unique Value and Frequency Counting

Solution Idea

A hash map is perfect for this exercise, since we need to save a key – value pair! The HashMap class in java has a neat method called merge that I used for this exercise. It just adds the value to the map, if it already exists it automatically updates the value!

Code

```
private HashMap<Integer,Integer> countFrequencies(int[] array)
{
```

```
    HashMap<Integer,Integer> frequencyHashMap = new HashMap<>();

    for (int value : array)
    {
        frequencyHashMap.merge(value, 1, Integer::sum);
    }
    return frequencyHashMap;
}
```

Figure 20: countFrequencies

Test

```
@Test
void testUniqueValueAndFrequencyCounting()
{
    int[] testArray = {5, 2, 8, 2, 5, 5, 1, 8};

    HashMap<Integer, Integer> expectedFrequency = new
HashMap<>();

    expectedFrequency.put(5, 3);
    expectedFrequency.put(2, 2);
    expectedFrequency.put(8, 2);
    expectedFrequency.put(1, 1);

    UniqueValueAndFrequencyCountingExercise exercise = new
UniqueValueAndFrequencyCountingExercise(testArray);
    exercise.runExercise();
    exercise.displayOutput();

    HashMap<Integer, Integer> actualFrequency =
exercise.getUniqueValuesFrequency();

    assertEquals(expectedFrequency, actualFrequency);
}
```

Figure 21: frequency test

```
=== Unique Values and Frequency ===
Original array: [5, 2, 8, 2, 5, 5, 1, 8]
Unique values and frequencies:
```

```
1 appears 1 time
2 appears 2 times
5 appears 3 times
8 appears 2 times
```

Figure 22: Frequency Output

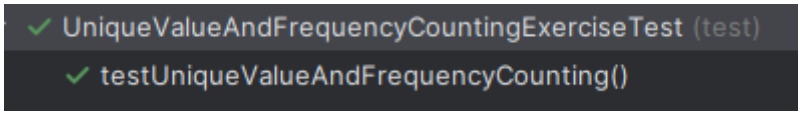

Figure 23: Frequency Unit test

Exercise 6 – Array Splitting

Solution Idea

An array only solution to this would be to first loop through the array and determine how many odd and even numbers there are, create the arrays and then then loop again and add the even and odd numbers to their arrays. I used lists instead, because it's cleaner.

Code

```
private ArraySplittingResult splitArray(int[] array)
{
    List<Integer> evenList = new ArrayList<Integer>();
    List<Integer> oddList = new ArrayList<Integer>();

    for (int num : array)
    {
        if (num % 2 == 0)
        {
            evenList.add(num);
        }
        else
        {
            oddList.add(num);
        }
    }
```

```
    int[] evenArray =
evenList.stream().mapToInt(Integer::intValue).toArray();
    int[] oddArray =
oddList.stream().mapToInt(Integer::intValue).toArray();

    return new ArraySplittingResult(evenArray, oddArray);

}
```

Figure 24: splitArray

Test

```
@Test
void testArraySplittingExercise()
{
    int[] inputArray = {15, 8, 23, 4, 42, 11, 19};

    int[] expectedEvenArray = {8, 4, 42};
    int[] expectedOddArray = {15, 23, 11, 19};

    ArraySplittingExercise splittingExcercise = new
ArraySplittingExercise(inputArray);

    splittingExcercise.runExercise();
    splittingExcercise.displayOutput();

    int[] actualEvenArray =
splittingExcercise.getArraySplittingResult().evenArray();
    int[] actualOddArray =
splittingExcercise.getArraySplittingResult().oddArray();

    assertArrayEquals(expectedEvenArray, actualEvenArray);
    assertArrayEquals(expectedOddArray, actualOddArray);
}
```

Figure 25: Array Splitting test

```
Array Splitting Exercise: Even and Odds
Original array: [15, 8, 23, 4, 42, 11, 19]
Even numbers: [8, 4, 42]
```

```
Odd numbers: [15, 23, 11, 19]
```

Figure 26: Array Splitting result

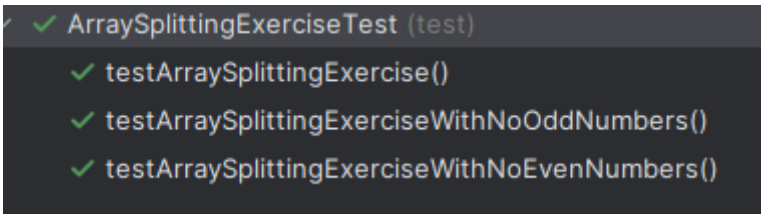

Figure 27: Array Splitting Unit Tests